Introduction

C++ and Visual Studio offer unparalleled value and versatility to developers worldwide. C++ is renowned for its efficiency, performance, and flexibility, making it a go-to language for building robust, high-performance applications across various domains. Its low-level features allow for precise control over system resources, making it ideal for developing resource-intensive applications like operating systems, games, and embedded systems.

Visual Studio, on the other hand, serves as a powerful integrated development environment (IDE) tailored specifically for C++ development, providing developers with a comprehensive suite of tools and features to streamline the software development process. Its intuitive interface, robust debugging capabilities, and seamless integration with C++ libraries make it the preferred choice for C++ developers seeking productivity and efficiency in their workflow.

In light of the pivotal role C++ and Visual Studio play in modern software development, these tools were used to implement a robust Library Management System for Stepwise University (A Terminal Based Program and Graphical Application). Efficient management of library resources is needed to facilitate seamless access to information for students, faculty, and staff. With C++ serving as the foundation for its implementation, alongside the comprehensive tools provided by Visual Studio, this software aims to harness the power of these technologies to deliver a sophisticated solution tailored to the specific needs of Stepwise University's library system.

Attention to detail is imperative in software engineering, and this project is no exception. By meticulously adhering to the project's requirements and specifications, we ensure that the Library Management System meets the expectations of its users while aligning with the university's workflow seamlessly whilst encapsulating advanced functionalities that are essential for library operations; particularly, book registration, borrowing, return, and fine calculation mechanisms.

Methodology

The methodology for developing the Library Management System for Stepwise University began with a meticulous requirement analysis. This involved breaking down the specified features into distinct user defined data types with member variables and functions; classes and delineating their interactions.[4] Subsequently, rough description of an algorithms were drafted to outline the variables and inputs required for method construction.[5] Extensive research was conducted to explore validation methods and error handling techniques to ensure the robustness of the system. With a clear understanding of the requirements, the program was developed to encompass the specified functionalities.

The specific requirements and functionalities expected from the Library Management System were meticulously defined. This encompassed tasks such as registering books, which could only be done by librarians, and involved storing details such as BookID, Title, Authors, Year, Publisher, and other relevant information in a CSV file. Borrowing and returning books were facilitated for both students and librarians, with borrowing limited to two weeks and borrowing details stored separately. Fines were calculated for overdue books, with fines recorded in a separate file. Non-functional requirements tightly aligned with object-oriented programming play an invaluable role in the successful delivery of this program. Some of which include; registering books for testing, exception handling, encapsulation, and input validation, were also delineated.

In the realm of technology selection, criteria such as performance, scalability, compatibility, and ease of development were considered. C++ was chosen as the primary programming language, leveraging its efficiency and flexibility, while Visual Studio was selected as the integrated development environment (IDE) for its comprehensive toolset.

```
RetBook::RetBook(SearchLibrary& sl, Program_Data& pd, Input_Validations& iv, f

void RetBook::returnaBook() {
    msg = "Enter The ID of the Borrower: ";
    user_id = iv.find_users_id(msg);

    cout << endl;

    returnaBook(user_id);
}

void RetBook::returnaBook(string user_id) {

    string msg = "Enter The Book Title or ID For This Return: ";
    search = iv.search_for_a_book(msg);
    cout << endl;

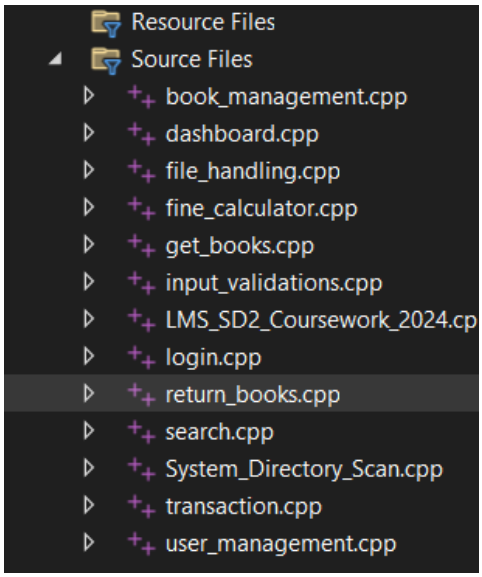
    if (search != "") {
        if (std::find(pd.sys_Users[user_id].currentBook.begin(), pd.sys_Users[
            retDate = iv.formattedDate();
            total_fees = fc.calculate_fines(user_id, retDate, search);
```

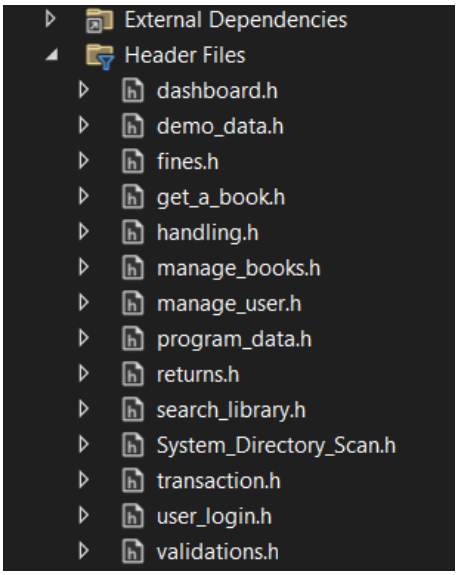
We optimized for performance by using overloading and rewriting code to be more efficient. Polymorphism is described as having many forms. This ideology provides a flexible mechanism for managing adaptive processes. In the case of our program, users borrowing or returning a book essentially user the same method. However, we are about to overload a function to capture missing data as a librarian user who may need to enter the customers ID for transactions.[6]

System design involved revisiting the initial program's architecture to enhance robustness and reliability. Attention was paid to usability aspects initially overlooked in the specifications. For instance, the implementation of system time replaced user-defined return dates to maintain data integrity. Limitations on borrowing to prevent stock depletion and the incorporation of a transaction module to prevent fee evasion were also addressed. System resilience features were added to detect file tampering, ensuring uninterrupted user experience.

The data model primarily utilized dynamic vectors and dictionaries, while the user interface design emphasized readability and user-friendliness through well-spaced prompts and tabulated information. System components, including header files such as iostream, string, unordered_map, and vector, were carefully selected to facilitate various operations within the program.

Consideration of principles of object-oriented design, such as encapsulation and interfaces, alongside extensibility through design patterns, ensured modularity and flexibility in the system. For instance, the implementation of vectors in storing book details allowed for future extensions to conduct searches via tags, keywords, and substrings.





Technologies Used:

User Interface:

- **.Net Framework**
- **Winforms**
- **iostream:** Provides functionality for input and output streams in C++. It includes classes like **cin** and **cout** for standard input and output operations.
- **string:** Provides functionality for working with strings in C++. It includes the **string** class for string manipulation operations.
- **iomanip:** Helps to manipulate input and output formatting, such as setting field width and precision, to enhance the readability of outputs.
- **cctype:** Includes functions for character handling in C++, useful for converting characters to lower or uppercase.
- **locale:** Changes the behavior of input operations like **cin**, eliminating the need for double enter presses and enhancing user interaction.

Data Handling:

- **unordered_map:** Provides an implementation of an associative container that stores elements formed by the combination of a key and a mapped value. Allows for fast retrieval of values based on keys.
- **vector:** Offers an implementation of a dynamic array, useful for storing sequences of elements that can grow or shrink in size.
- **sstream:** Facilitates working with string streams, allowing strings to be treated as input/output streams.
- **algorithm:** Provides a collection of functions for performing common algorithms and operations on sequences, such as sorting and searching.

File Handling:

- **fstream:** Includes classes for working with files in C++, including classes for reading from and writing to files.
- **windows.h:** Specific to Windows, it provides functions and macros for working with the Windows API. Used to launch the receipt file into an open window.

Error Handling:

- **EOF Handling:** Refers to the process of detecting when the end of an input stream has been reached. Implemented to handle end-of-file conditions gracefully.
- **Exception Handling:** A mechanism in C++ for dealing with errors and other exceptional conditions that occur during program execution. Try-catch blocks were used to handle exceptions effectively.

Programming Concepts:

- **Classes:** Fundamental feature of object-oriented programming (OOP) in C++, used to encapsulate data and behavior into objects, promoting modularity and code reuse.
- **Encapsulation:** Principle of OOP involving bundling data and methods that operate on the data into a single unit (i.e., a class), helping to hide the internal state of an object and restrict access to it from outside the class.

Validations:

- **Regular Expressions (RegEx):** Utilized for pattern matching and validation of input data.
- **Try-Catch Blocks:** Implemented to handle exceptions and errors gracefully during program execution.
- **EOF Handling:** Used to detect end-of-file conditions during file input operations.

```cpp
# include <iostream>
# include <string>
# include <unordered_map>
# include <vector>
# include <iomanip> // just to help format the outpu
# include <cctype> // Include the cctype header for
# include <locale> // This changes the behaviour of
# include <ctime>
# include <sstream>
# include <algorithm>
# include <fstream> // dealing with files and writti
# include <regex>

using std::cout;
using std::cin;
using std::endl;
using std::string;
using std::unordered_map;
using std::vector;
using std::setw;

using namespace std;
```

```cpp
#pragma once
#include <windows.h>
#include <string>

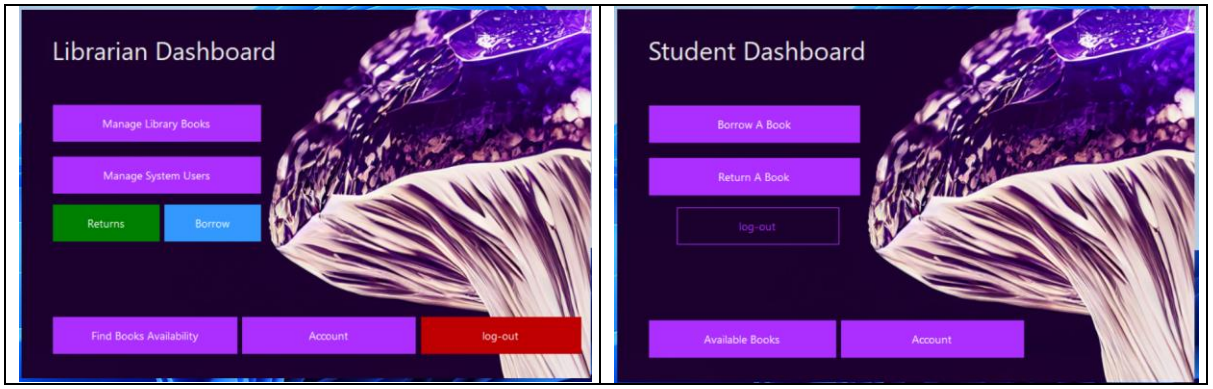
// We are going to note the operating system.
// I am going to attempt to call an action for

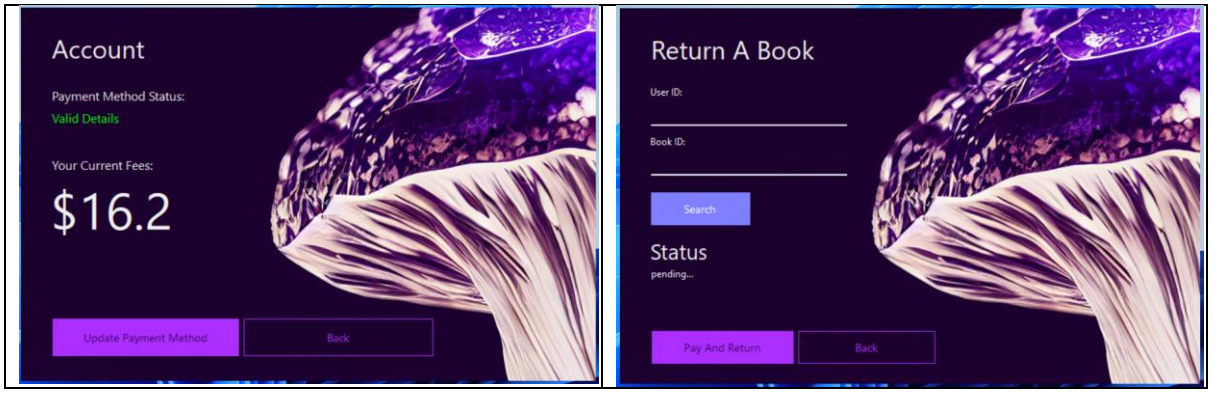
// Check if the operating system is Windows
#if defined(_WIN32) || defined(_WIN64)
const std::string os = "Windows";
// Check if the operating system is macOS
#elif defined(__APPLE__)
const std::string os = "macOS";
// Check if the operating system is Linux
#elif defined(__linux__)
const std::string os = "Linux";
#else
const std::string os = "Unknown";
#endif

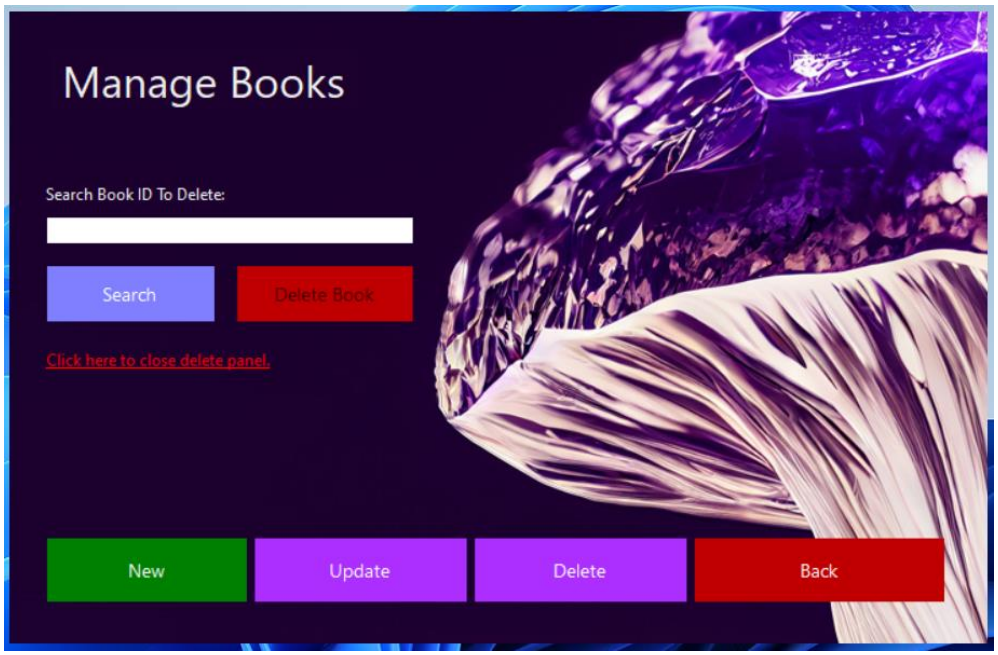
#include <cstdlib>
```

Overall, the testing phase confirmed that the system met the defined requirements and objectives, demonstrating its reliability and performance in real-world scenarios.

Debugging and testing, as essential components of the development process, played a crucial role in ensuring the system's correctness and reliability. Various debugging techniques, including setting breakpoints, stepping through code, inspecting variables, and utilizing debugging tools provided by Visual Studio, were employed to identify and resolve software issues effectively.







Conclusion:

In conclusion, the project has achieved its objectives of developing a robust and reliable system for library management. Through the implementation of effective debugging and testing practices, the system's functionality has been thoroughly validated, ensuring its correctness and reliability.

The documentation produced throughout the development process providing valuable insights into the system's design, implementation, and testing procedures.

Exception handling, has been effectively implemented to handle errors and exceptional conditions that may arise during program execution. By incorporating try-catch blocks, EOF handling, regular expressions, and other techniques, the system is able to gracefully handle unexpected scenarios, enhancing its robustness and reliability.

Looking ahead, future work may involve further enhancements to the system, such as incorporating additional features based on user feedback, improving system performance and scalability, and refining the user interface to enhance usability. By continuing to iterate and refine the system based on user feedback and evolving requirements, the system can remain relevant and effective in meeting the needs of its users.